



RTL Design Sherpa

DDR2/LPDDR2 Characterization Nexys A7 Project Guide 0.90

July 2026

Table of Contents

1 Document Information.....	8
1.1 References.....	8
1.2 Terminology.....	9
1.3 Revision History.....	9
2 Overview.....	10
2.1 What it characterizes.....	10
2.2 The three flows.....	10
2.3 Board and tools.....	11
2.4 Workload families.....	11
3 How It Works.....	12
3.1 The transport spine.....	12
3.1.1 Figure 2.1: DDR2 Characterization Harness Spine.....	12
3.2 The 1→4 AXIL bridge.....	12
3.3 Key RTL blocks.....	12
3.4 The characterization engines.....	13
3.5 The controller, DFI, and PHY (black boxes).....	14
4 Test Methodology.....	14
4.1 What is under test.....	14
4.2 What is measured, and how.....	14
4.3 Workloads.....	15
4.4 The oracle.....	15
4.5 Measurement pitfalls.....	15

5 Build and Run.....	16
5.1 build-perf make targets.....	16
5.2 The host programs (build-perf/host/).....	17
5.3 The performance sweeps (build-perf/bin/).....	18
5.4 Typical bring-up sequence.....	19
6 Harness CSR Configuration.....	19
6.1 Bridge address map.....	19
6.2 By-name access (RDL → regmap).....	20
6.2.1 Waveform 5.1: Engine Kick and Poll.....	20
6.3 Control / status (offsets relative to HARNESS_CSR_BASE).....	20
6.4 Timer.....	22
6.5 Runtime controller cfg + DFI tuning.....	22
6.6 a7ddrphy calibration passthrough (hardware only).....	23
6.7 Engine config (WR 0x100–0x128, RD 0x180–0x1A8).....	23
6.8 Perf observability (0x1C0–0x1E8).....	24
7 Simulation / Silicon Equivalence.....	25
7.1 Testbenches.....	25
7.2 How the same program runs in sim.....	26
7.3 Match the board’s exact DFI config.....	26
7.4 What the sim can and cannot reproduce.....	26
8 Troubleshooting.....	26
8.1 Current board state.....	26
8.2 Sticky errors look like a wedge.....	27
8.3 Known bugs / bounds.....	27

8.4 Cannot find the board.....	27
8.5 a7ddrphy is not simulatable.....	27
8.6 debug_sram trace ring is disabled on this build.....	28
8.7 LiteDRAM baseline flow.....	28
9 Measured Results.....	28
9.1 8.1 Streaming bandwidth, and why short bursts fall short.....	28
9.2 8.2 How much idle the controller absorbs.....	29
9.2.1 Figure 8.1: Read Bandwidth vs Gap, Row-Major.....	30
9.2.2 Figure 8.2: Read Bandwidth vs Gap, Cacheline.....	31
9.2.3 Figure 8.3: Read Bandwidth vs Gap, Col-Major.....	31
9.2.4 Figure 8.4: Bus Bandwidth Across Gap and Generator Count.....	32
9.3 8.3 Where the cycles went.....	32
9.3.1 Figure 8.5: Cycle Breakdown at 1+1 — the Bend and its Cause.....	33
9.3.2 Figure 8.6: Cycle Breakdown at 4+4 — Saturated at Every Gap.....	34
9.4 8.4 Integrity, and one configuration that must not be quoted.....	35
9.5 8.5 Reproducing these numbers.....	36

List of Figures

Figure 2.1: DDR2 Characterization Harness Spine.....	12
Figure 8.1: Read Bandwidth vs Gap, Row-Major.....	30
Figure 8.2: Read Bandwidth vs Gap, Cacheline.....	31
Figure 8.3: Read Bandwidth vs Gap, Col-Major.....	31
Figure 8.4: Bus Bandwidth Across Gap and Generator Count.....	32
Figure 8.5: Cycle Breakdown at 1+1 — the Bend and its Cause.....	33
Figure 8.6: Cycle Breakdown at 4+4 — Saturated at Every Gap.....	34

List of Tables

Table 1: Reference documents.....	8
Table 2: Revision history.....	9
Table 3: The characterization flows.....	10
Table 4: Board and tool requirements.....	11
Table 5: Access-pattern families.....	11
Table 6: Key RTL blocks.....	12
Table 7: What is measured.....	14
Table 8: Characterization workloads.....	15
Table 9: build-perf make targets.....	16
Table 10: Bridge slave address map.....	19
Table 11: Control / status registers.....	20
Table 12: Characterization timer registers.....	22
Table 13: Runtime controller config and live DFI tuning.....	22
Table 14: a7ddrphy calibration passthrough.....	23
Table 15: Write-engine config (read engine identical at +0x80).....	24
Table 16: Perf observability (bin b covers $[2^b, 2^{(b+1)})$ cycles).....	24
Table 17: Simulation testbenches.....	25
Table 18: Known issues and bounds.....	27
Table 19: Read bandwidth against AXI burst length, at 8 bursts outstanding.....	28
Table 20: The short-burst shortfall recovers completely when more reads are in flight.....	29
Table 21: Gap knee by address order and generator count.....	30
Table 22: Read cycle breakdown, row-major, 1+1 generators.....	33
Table 23: PUMICE-037 — narrowing the failing configuration.....	35

List of Waveforms

Waveform 5.1: Engine Kick and Poll.....	20
---	----

1 Document Information

This is the operator / developer guide for the Nexys A7 **DDR2/LPDDR2 Characterization** project (projects/fpga-systems/NexysA7/pumice/ddr2-characterization). It explains what the project characterizes, how the harness works, how to build / simulate / program / run it, how the harness CSR is configured, and what the board measured (Chapter 8, added at v0.95).

The memory controller under test — **pumice** — is separate IP (projects/components/memory-controllers/pumice-ddr2-lpddr2). This guide treats it as a **black box** driven by the characterization harness; it does not document the controller’s internals.

Note: the project’s top-level README.md still carries an early “skeleton — RTL not yet written” status line. That is stale: the harness, host, sim, and both flows are built and running. Trust this guide and the RTL/host sources over that status.

1.1 References

Reference documents

Source	Title
RTL Design Sherpa	build-perf/host/ADDRESS_MAP.md (bridge + CSR map)
RTL Design Sherpa	build-perf/bin/README_a7ddrphy.md (PHY regen)
RTL Design Sherpa	flows-litedram-uart/HARNESS_PLAN.md (baseline flow)
RTL Design Sherpa	bin/TBClasses/harness/ (shared UART-char collateral)
RTL Design Sherpa	docs/DDR2_BANDWIDTH_MEASUREMENT.md (what “bandwidth” means here)
RTL Design Sherpa	build-perf/reports/ (the raw

Source	Title
	records behind Chapter 8)
Micron	MT47H64M16HR - 25E DDR2 SDRAM datasheet
ARM	AMBA AXI4 / AXI4-Lite / APB Protocol Specifications

1.2 Terminology

DFI

DDR PHY Interface. The command/data protocol between the memory controller and the PHY. Here pumice emits a phase-packed flat DFI v2.1 bus.

a7ddrphy

LiteDRAM's Artix-7 DDR2 PHY (4:1 serdes, 800 Mbps). Uses Xilinx OSERDESE2/ISERDESE2/IDELAYE2 and therefore does **not** simulate in Verilator.

GEAR / DFI_RATE

The DFI phase count. $\text{GEAR_RATIO} = \log_2(\text{DFI_RATE})$. Must match between the RTL build and the sim model or reads corrupt.

Engine

A master-side AXI4 traffic generator: the write engine emits an LFSR data pattern; the read engine re-reads and CRC-checks it.

Leveling

Firmware-driven a7ddrphy read/write training over the PHY-CSR passthrough window; there is no hardware leveling FSM.

1.3 Revision History

Revision history

Version	Date	Notes
0.90	2026-07-18	Initial project guide

2 Overview

2.1 What it characterizes

This project performs end-to-end, board-level **performance, latency, and data-integrity characterization** of a DDR2/LPDDR2 memory controller on real silicon. It drives the onboard Micron MT47H64M16HR - 25E DDR2 (x16, single-rank, 128 MiB, up to 800 Mbps) through the controller against the real device, and reports throughput (MB/s), latency histograms, and per-workload CRC integrity across burst-length / stride / gap / page-policy / scheme sweeps.

There is no soft CPU in the FPGA — an off-board Python **host** drives the harness over the FTDI USB-UART (115200 8N1). The same host programs run against a cocotb simulation (Chapter 6).

2.2 The three flows

The project is organized as sibling “flows” plus a shared framework:

The characterization flows

Flow	Controller (DUT)	What it is
build-perf/	pumice DDR2 (black box) + DFI + a7ddrphy	Primary flow: UART-driven host + Vivado build of the pumice harness on the Nexys A7.
flows-litedram-uart/	LiteDRAM litedram_core	Apples-to-apples baseline: LiteDRAM’s own DDR2 controller (self-initializing, own PLL/PHY) driven by the same engines/taps/host.
ddr2_char_framework/	—	Shared instrumentation: harness RTL, the engine spine (char_engine_block) and its pumice wrapper (ddr2_char_macro), the

Flow	Controller (DUT)	What it is
		1→6 AXIL bridge, and the cocotb DV.

The naming convention is flows-<controller>-uart/ for on-Nexys builds (host over UART, no CPU). pumice is separate IP under projects/components/memory-controllers/pumice-ddr2-lpddr2/.

2.3 Board and tools

Board and tool requirements

Item	Detail
Board	Digilent Nexys A7-100T, Artix-7 xc7a100tcsq324-1
DRAM	Onboard Micron MT47H64M16 DDR2 x16, ROW_WIDTH = 13
Build	Vivado (batch mode)
Host	Python 3 + pyserial
Sim	Verilator + cocotb / cocotb_test / pytest
PHY regen (only)	Python 3.10 + migen / litex 2024.12 / litedram 2024.12

2.4 Workload families

The characterization sweeps four access-pattern families (build-perf/host/pumice_char.py):

Access-pattern families

Family	Access	Purpose
incremental	contiguous linear	best-case sequential
row_major	bounded to one page	guaranteed page HIT
col_major	stride = one row, same bank	guaranteed page MISS (worst case)
col_major_interleaved	stride = one bank	activates the bank pipeline

Controller-config presets (baseline, bank_interleave, open_page, inorder, reorder, happy_hybrid, fast_refresh, slow_refresh) each flip one lever from baseline; they are driven over the pumice APB slave (black box). Run profiles smoke / matrix / full are shared by sim and board.

File	Role
build-perf/rtl/ ddr2_char_harness.sv	Internal integration: uart_axil_bridge + bridge_ddr2_char_axil + harness_csr + debug_sram + dfi_mon_ram + ddr2_char_macro + char timer + LED / 7-seg.
ddr2_char_framework/rtl/ char_engine_block.sv	The DUT-agnostic spine: chargen_regs (per-generator config behind the chargen_apb window), the generator arrays (axi4_master_wr_pattern_gen / axi4_master_rd_crc_check), the two per-direction crossbars that merge them onto one AXI4 master, and the perf taps (bus meters + latency histograms) on that port. Shared by the pumice and LiteDRAM flows.
ddr2_char_framework/rtl/ ddr2_char_macro.sv	Wraps char_engine_block around pumice: the block's AXI4 master drives the controller's s_axi, and the APB CSR / DFI / runtime controls pass through.
ddr2_char_framework/rtl/ harness_csr.sv	The AXIL CSR slave (Chapter 5). Hand-written (self-clearing pulses, latches, PHY passthrough).
ddr2_char_framework/rtl/ dfi_v21_flat_to_a7ddrphy.sv	Combinational adapter: pumice's phase-packed flat DFI v2.1 → a7ddrphy per-phase ports (NPHASES = 4).
ddr2_char_framework/rtl/ a7ddrphy_stub.sv	Port-shape black box of a7ddrphy for Verilator (real body swapped in by Vivado).

3.4 The characterization engines

Inside char_engine_block (instantiated by ddr2_char_macro, and by the LiteDRAM flow's char_engine_harness), master-side AXI4 engines generate and check traffic against the controller's s_axi port:

- **Write engine** (`axi4_master_wr_pattern_gen`) — emits an LFSR data pattern across the programmed address pattern, and latches an expected CRC.
- **Read engine** (`axi4_master_rd_crc_check`) — re-reads the same pattern and computes an actual CRC, plus a per-beat mismatch count.

Alongside them, **perf taps** — `axi_bus_meter` (write and read) and `axi_perf_latency_hist` — observe the internal AXI wires to produce the producer/backpressure/starve/idle buckets and the latency histogram read back in Chapter 5.

3.5 The controller, DFI, and PHY (black boxes)

The controller (**pumice**) sits inside `ddr2_char_macro` between the AXI engines and the flat DFI output. Its configuration registers are reached over the `ddr2_apb` bridge slave at base `0x0`. This guide documents only that the harness drives it — not its internals.

`pumice`'s flat DFI v2.1 bus is adapted to the `a7ddrphy` per-phase interface, then to the DDR2 pads. **a7ddrphy does not simulate in Verilator** (it uses Xilinx SERDES/IDELAY primitives); the sim connects at the DFI level with a behavioral model instead (Chapter 6). On hardware, `a7ddrphy` calibration is driven by firmware (the host) over the PHY-CSR passthrough window — there is no hardware leveling FSM.

4 Test Methodology

This chapter describes what is exercised on the FPGA block (Figure 2.1) and how throughput, latency, and data integrity are measured.

4.1 What is under test

The **pumice** memory controller (black box) driving the real onboard DDR2, stimulated by the two master-side AXI4 engines. Every measurement is taken on the internal AXI wires between the engines and the controller, so the numbers reflect the controller + PHY + device as a whole.

4.2 What is measured, and how

What is measured

Quantity	How it is obtained
Throughput	$\text{beats} \times \text{bytes_per_beat} / (\text{TIMER_CYCLES} \times 10 \text{ ns})$ — the timer bounds the run in <code>sys_clk</code> cycles
Latency histogram	<code>OBS_HIST_SEL</code> selects bus (rd/wr) + metric (<code>AR→firstR</code> / <code>AR→RLAST</code>) + bin; <code>OBS_HIST_COUNT</code> / <code>OBS_HIST_TOTAL</code> read it back. Bin <i>b</i> covers $[2^b,$

Quantity	How it is obtained
	$2^{(b+1)}$ cycles
Data integrity	CRC_ACTUAL vs CRC_EXPECTED (CRC_MATCH), plus BEATS_MISM
Utilization	OBS_{RD,WR}_{PROD,BP,STARV,IDLE} bus-meter buckets

A measurement run: program the engine config (address pattern, burst length, txn count, LFSR seed), `clear_stats`, kick with `CTRL.start_wr / start_rd`, poll `STATUS.wr_done / rd_done`, then read the timer, CRC, histogram, and meters. The write engine latches the expected CRC as it generates the pattern; the read engine re-reads and computes the actual CRC.

4.3 Workloads

Characterization workloads

Axis	Values
Access pattern	incremental, row_major, col_major, col_major_interleaved
Controller preset	baseline, bank_interleave, open_page, inorder, reorder, ... (over the pumice APB slave)
Burst / count / stride	swept per --char-profile (smoke / matrix / full)

4.4 The oracle

Integrity is judged against a deterministic **LFSR pattern + CRC-32**: the write engine's CRC_EXPECTED must equal the read engine's CRC_ACTUAL with BEATS_MISM = 0. The same golden logic runs in sim (Chapter 6).

4.5 Measurement pitfalls

The full treatment — the four candidate denominators, how to choose the measurement window, the bounds a number should be checked against, and the complete pitfall list — is `docs/DDR2_BANDWIDTH_MEASUREMENT.md` in this area. Read it before publishing a bandwidth figure. What follows is only the operational short list.

- `STATUS.any_error` is **sticky** — clear it with `clear_stats` between phases or every later run looks wedged.

- **The mismatch counter is PER READER.** `beats_mismatched()` with no argument checks reader 0 only; on a 4+4 run that leaves three readers unverified and reports them clean. Sum across the active engines.
- **Sweep points are not independent unless you re-fill.** A point that leaves wrong data in cells is inherited by every later point that reads the region, and the inherited damage is reported as the later point's own result. This made the first 192-point gap sweep unreadable (PUMICE-037).
- **A constant repeated across runs is stale state, not a race.** A race varies. An identical mismatch count at eight consecutive gaps means one artifact is being re-read eight times.
- **Writers and readers on disjoint banks means nobody verifies the writers' banks.** A defect can sit there for a whole run and surface only when a later configuration happens to put a reader on a bank earlier points wrote — which is exactly how PUMICE-037 appeared, in the last block of a 192-point sweep.
- Keep board runs $\text{txn} \leq 1024$ (read-engine CAM ceiling) for clean completion; the *rate* stays valid at scale.
- `col_major stride` $\times$ large `txn` can alias past the 128 MB device — bound it.
- The sim must be built at the board's exact `DFI_RATE / GEAR` or it passes while the board fails (Chapter 6).

5 Build and Run

All commands assume the Python environment is sourced (sets `REPO_ROOT`, `SIM=verilator`, `PATH`, `PYTHONPATH`):

```
cd /path/to/RTLDesignSherpa && source env_python
cd projects/fpga-systems/NexysA7/pumice/build-perf
```

5.1 build-perf make targets

`UART` `?` `/dev/ttyUSB1`, `BAUD` `?` `115200`, `VIVADO` `?` `vivado`.

build-perf make targets

Target	Action
<code>make lint</code>	Verilator lint of <code>ddr2_char_top</code> via the harness filelist
<code>make project / make synth</code>	create Vivado project / synth-only +

Target	Action
	reports
make bitstream	full synth + impl + bitgen (~10–30 min) → bitstream/ddr2_char.bit
make bitstream-ila	same, plus an ILA on the DFI boundary → .bit + .ltx
make program	flash the board
make utilization/make timing	print the latest reports
make smoke	host/run_smoke.py — link + one linear WR/RD integrity pass
make status	host/ddr2_char.py — one-shot BUILD_ID + status dump
make level	host/pumice_master.py --level- only — a7ddrphy leveling
make simple	host/pumice_master.py --simple — init + one write→read pass
make characterize	host/pumice_master.py --full — full workload sweep
make sweep-rddly/make train- deskew	PHY timing sweeps
make host-test	pytest host/test_pumice_master.py (mock UART, no board)
make sim/sim-smoke/sim-simple/ sim-level	run the same host programs in cocotb sim
make clean/clean-all	remove build artifacts

The other Makefiles: flows-litedram-uart/Makefile (make regen/make bitstream/make program for the LiteDRAM baseline) and ddr2_char_framework/dv/tests/Makefile (the cocotb macro suite: make run, run-smoke, run-<shape>, and parallel run-{gate,func,full}-parallel).

5.2 The host programs (build-perf/host/)

The port defaults to `--port auto`, which probes every `/dev/ttyUSB*` and keeps the board that answers `BUILD_ID == 0x44445232` (“DDR2”).

run_smoke.py — the first thing to run after flashing. Programs a linear WR+RD workload, kicks both engines, and checks CRC / cycles / perf.

```
python3 host/run_smoke.py --port /dev/ttyUSB1 --txn 1024 --blen 8 --seed 0xDEADBEEF
```

Pass = BUILD_ID matches, both engines done, CRC_ACTUAL == CRC_EXPECTED, BEATS_MISMATCHED == 0, and TIMER.pass.

pumice_master.py — the orchestration program; one mode is required:

```
python3 host/pumice_master.py --port /dev/ttyUSB1 --level-only # PHY read/write leveling
python3 host/pumice_master.py --port /dev/ttyUSB1 --simple # init + one write→read pass
python3 host/pumice_master.py --port /dev/ttyUSB1 --full # full workload sweep
python3 host/pumice_master.py --port /dev/ttyUSB1 --char --char-profile matrix --char-scale 1000 --csv out.csv
```

Useful options: --no-level, --rd-phase, --rd-delay, --char-configs, --char-scale N (≈1000 on FPGA), --level-cache JSON, --clk-mhz.

ddr2_char.py — the driver library plus a one-shot status CLI (python3 host/ddr2_char.py --port auto). It exports DDR2CharDriver, the base-address constants, and the enum constants used throughout.

5.3 The performance sweeps (build-perf/bin/)

Three scripts, each isolating one axis. Run them from build-perf/ after make program; all three write JSON or a table and none need arguments.

axlen_sweep.py — bandwidth against AXI burst length at a fixed outstanding budget. This is the headline read/write figure and the Little's-law fit.

outstanding_sweep.py — bandwidth against the number of bursts in flight, at a fixed AxLEN. This is the DIRECT test of the latency-bound claim: if a shortfall is Little's law, the knee sits near latency/AxLEN and moves as 1/AxLEN. A knee well before that names whatever the real limit is instead. The runtime max_outstanding dial and the 32-deep ceiling exist for this sweep.

bank_gap_sweep.py (plot with plot_bank_gap.py) — N writers and N readers running CONCURRENTLY, one bank apiece, across inter-burst gap 0..15, three address orders, at 4+4 / 3+3 / 2+2 / 1+1 engines. It answers how much idle the controller absorbs before bandwidth falls.

```
python3 bin/axlen_sweep.py
python3 bin/outstanding_sweep.py
python3 bin/bank_gap_sweep.py && python3 bin/plot_bank_gap.py
reports/bank_gap_sweep.json
```

Useful env: GENS, GAPS (a named sweep — full, knee, ends — or a literal list), TXN, and PREFILL. **Leave PREFILL at its default point.** It re-fills the whole device before every measurement so points are independent; PREFILL=once exists only for a deliberate damage-accumulation

experiment, not to go faster. The gap range stops at 15 because the CSR field is four bits — asking for 16 programs 0 and silently re-measures back-to-back under a different label.

Reading the output: the knee column is the largest gap still within 3% of the gap-0 value. A FLAT curve at high generator counts is a result, not a broken axis — a gap only bends the curve once aggregate demand drops below what the controller can deliver, and four generators at gap 15 are still over-subscribing it. Points whose data failed verification are ringed in the plots and listed as FAIL: lines with their stray-beat count; their bandwidth is still a real measurement, but the configuration is not a clean operating point.

5.4 Typical bring-up sequence

```
make bitstream
make program
make smoke    UART=/dev/ttyUSB1      # link + integrity
make level    # find the read/write eye
make simple   # one clean write→read
make characterize # full sweep -> CSV
# no board? prove the same programs in sim:
make sim
```

6 Harness CSR Configuration

6.1 Bridge address map

The UART bridge fans out to four AXI4-Lite slaves. Bases come from `bridge_ddr2_char_axil.toml` (mirrored in `build-perf/host/ADDRESS_MAP.md` and as constants in `ddr2_char.py`):

Bridge slave address map

Constant	Slave	Base	Size	Protocol
DDR2_APB_BASE	ddr2_apb	0x0000_0000	4 KB	APB — pumice controller CSR (black box)
HARNESS_CSR_BASE	harness_csr	0x0001_0000	4 KB	AXIL — this chapter
DEBUG_SRAM_BASE	debug_sram	0x0004_0000	256 KB	AXIL 64b — MonBus/DFI trace ring
DFI_MON_RAM_BASE	dfi_mon_ram	0x0008_0000	4 KB	AXIL — DFI cmd observability

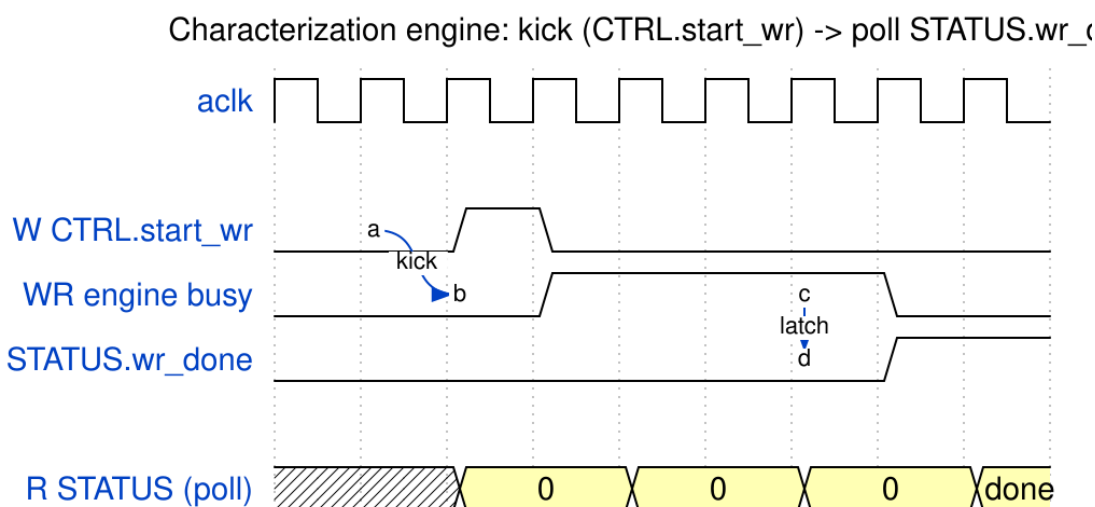
6.2 By-name access (RDL → regmap)

harness_csr.sv is hand-written; harness_csr.rdl is a descriptor of it, from which the by-name host regmap is generated. Registers are accessed by name via UartRegisterMap, never by hardcoded offset. Regenerate after editing either:

```
python3 bin/peakrdl_generate.py \  
  
projects/fpga-systems/NexysA7/pumice/ddr2_char_framework/rtl/harness_csr.rdl \  
    --regmap --docs-only --no-html --no-markdown \  
    --regmap-output \  
projects/fpga-systems/NexysA7/pumice/ddr2_char_framework/dv/tbclasses/harness_csr_regmap.py
```

A consistency test (build-perf/host/test_harness_regmap_consistency.py) asserts the generated regmap matches the SV header table.

6.2.1 Waveform 5.1: Engine Kick and Poll



Engine kick and poll

Source: [01_engine_kick.json](#)

6.3 Control / status (offsets relative to HARNESS_CSR_BASE)

Control / status registers

Register	Offset	Field	Bits	Access	Description
CTRL	0x00	start_wr	[0]	W	Pulse: kick the write engine

Register	Offset	Field	Bits	Access	Description
CTRL	0x00	start_r d	[1]	W	Pulse: kick the read engine
CTRL	0x00	clear_s tats	[2]	W	Pulse: clear perf + sticky errors
CTRL	0x00	freeze_ trace	[3]	RW	Latch: freeze perf counters
CTRL	0x00	soft_re set	[4]	W	Pulse: soft reset (does not clear sticky errors)
STATUS	0x04	wr_done / rd_done	[0] / [1]	R	Engine completion
STATUS	0x04	wr_erro r / rd_erro r	[2] / [3]	R	Per-engine error
STATUS	0x04	any_err or	[4]	R	Sticky OR of errors — cleared by clear_stats
STATUS	0x04	init_do ne / init_fa il	[6] / [7]	R	Controller init status
DBG_WR_PTR	0x08	value	[31:0]	R	Words written to debug_sram since clear
DBG_OVERFLOW	0x0C	overflow	[0]	R	Sticky trace-overflow
CRC_EXPECTED	0x10	value	[31:0]	R	Write-engine expected CRC
CRC_ACTUAL	0x14	value	[31:0]	R	Read-engine actual CRC
CRC_MATCH	0x18	match	[0]	R	CRC_ACTUAL == CRC_EXPECTED
CRC_MATCH	0x18	exp_val id /	[1] / [2]	R	CRC captured

Register	Offset	Field	Bits	Access	Description
		act_val id			
CRC_MATCH	0x18	beats_m ism_nz	[3]	R	Any beat mismatched
SCRATCH	0x1C	value	[31:0]	RW	Host link ping
BUILD_ID	0x20	value	[31:0]	R	0x44445232 (“DDR2”)
BEATS_MISM	0x24	value	[31:0]	R	Beats mismatched

6.4 Timer

Characterization timer registers

Register	Offset	Bits	Access	Description
TIMER_CTRL	0x28	[0] clear	W	Pulse: clear the cycle counter
TIMER_STATUS	0x2C	[0] done, [1] running, [2] pass	R	Timer state
TIMER_CYCLES_L 0/HI	0x30 / 0x34	[31:0]	R	64-bit cycle count (10 ns/cycle)
TIMER_EXP_BEAT S	0x38	[31:0]	RW	Beat-count stop trigger (0 = disable)
TIMER_{R,W}_{F IRST, LAST}_{LO , HI}	0x40–0x5C	[31:0]	R	First/last R/W beat timestamps (64-bit each)

6.5 Runtime controller cfg + DFI tuning

Runtime controller config and live DFI tuning

Register	Offset	Field	Bits	Access	Description
CTRLR_CFG	0x60	memtype	[0]	RW	0 = DDR2, 1 = LPDDR2
CTRLR_CFG	0x60	t_phy_w rlat	[15:8]	RW	PHY write latency (see Chapter 7 —

Register	Offset	Field	Bits	Access	Description
					board wants 0)
CTRLR_CFG	0x60	t_rddat a_en	[23:16]	RW	Read-data-enable latency
CTRLR_CFG	0x60	rd_in_o rder	[24]	RW	Force in-order reads
CTRLR_CAP	0x64	cap_loo kahead_ max	[3:0]	RW	Advertised OOO look-ahead depth
CTRLR_CAP	0x64	cap_syn th_mask	[7:4]	RW	Advertised feature mask
DFI_TUNING	0x68	cmd_del ay	[3:0]	RW	Live DFI cmd delay (reset 1) — no rebuild
DFI_TUNING	0x68	rddata_ delay	[7:4]	RW	Live DFI read-data delay (reset 0)

6.6 a7ddrphy calibration passthrough (hardware only)

Indirect access to the a7ddrphy's read/write-leveling knobs; firmware-driven, no HW FSM.
Meaningless in sim (the stub ignores it). Knob map:
rtl-vivado/a7ddrphy/a7ddrphy_csr_map.txt.

a7ddrphy calibration passthrough

Register	Offset	Bits	Access	Description
PHY_CSR_ADDR	0x80	[9:0]	RW	a7ddrphy CSR word index
PHY_CSR_WDATA	0x84	[31:0]	RW	Value to write
PHY_CSR_CTRL	0x88	[0] pulse	W	Drive one CSR-bus write
PHY_CSR_RDATA	0x8C	[31:0]	R	dat_r for the current PHY_CSR_ADDR

6.7 Engine config (WR 0x100–0x128, RD 0x180–0x1A8)

The read block mirrors the write block at +0x80.

Write-engine config (read engine identical at +0x80)

Register	Offset	Field	Bits	Description
WR_START_ADDR	0x100	value	[31:0]	Start address
WR_STRIDE_0/1	0x104 / 0x108	value	[23:0]	Signed strides
WR_WRAP_MASK_0 /1	0x10C / 0x110	value	[31:0]	Address wrap masks
WR_BLEN_TXN	0x114	burst_len	[7:0]	AXI burst length
WR_BLEN_TXN	0x114	txn_count	[23:8]	Transaction count
WR_BLEN_TXN	0x114	gap	[27:24]	Inter-burst gap
WR_AXI_ATTR	0x118	axi_id	[7:0]	AXI ID
WR_AXI_ATTR	0x118	id_mode	[9:8]	0 fixed / 1 counter / 2 LFSR
WR_AXI_ATTR	0x118	axi_size	[12:10]	AXI size (AXI_SIZE_8 = 3)
WR_AXI_ATTR	0x118	axi_burst	[14:13]	0 fixed / 1 incr / 2 wrap
WR_AXI_ATTR	0x118	data_mode	[15]	Data pattern mode
WR_LFSR_SEED	0x11C	value	[31:0]	Pattern seed (must match RD for CRC)
WR_HASH_SEED0/ 1/2	0x120–0x128	value	[31:0]	Hash seeds

6.8 Perf observability (0x1C0–0x1E8)

All 32-bit; cleared by CTRL.clear_stats, frozen by CTRL.freeze_trace.

Perf observability (bin b covers $[2^b, 2^{(b+1)})$ cycles)

Register	Offset	Field	Bits	Description
OBS_RD_{PROD,B P,STARV,IDLE}	0x1C0– 0x1CC	value	[31:0]	Read data-channel meter buckets
OBS_WR_{PROD,B P,STARV,IDLE}	0x1D0– 0x1DC	value	[31:0]	Write meter buckets
OBS_HIST_SEL	0x1E0	bus	[0]	0 = read, 1 = write
OBS_HIST_SEL	0x1E0	metric	[1]	0 = AR→firstR /

Register	Offset	Field	Bits	Description
				AW→B, 1 = AR→RLAST
OBS_HIST_SEL	0x1E0	bin	[5:2]	Histogram bin index (0–15)
OBS_HIST_COUNT	0x1E4	value	[31:0]	Count in the selected bin
OBS_HIST_TOTAL	0x1E8	value	[31:0]	Total txns on the selected metric

7 Simulation / Silicon Equivalence

This project follows the shared UART-characterization methodology (bin/TBClasses/harness/): **one authored-once host program runs byte-for-byte identically on the FPGA and in a cocotb simulation.** The equivalence boundary is the ASCII W/R byte stream on the UART wire; only the bottom ByteChannel differs — SerialChannel (pyserial) on the board vs CocotbUartChannel on the DUT pins in sim.

7.1 Testbenches

Simulation testbenches

tb_top	Wraps	Drives cfg via
ddr2_char_uart_t b_top.sv	the full ddr2_char_harn ess (real UART bridge + harness_csr + 1→N bridge + engines + pumice)	the real UART pins (cocotb UARTMaster/Monitor)
ddr2_char_macro_ tb_top.sv	the inner ddr2_char_macr o	direct ports + an APB BFM

The UART tb is the equivalence vehicle: it exposes real UART pins so the sim consumes the identical byte stream the host sends to silicon. The DFI side is wired to internal phy_dfi_* nets so the framework's DFISlavePHY + MemoryModel loopback auto-binds by prefix. **There is no a7ddrphy in sim** — it reproduces the digital DFI handshake, not the analog eye.

7.2 How the same program runs in sim

`test_ddr2_char_uart.py` builds the DFISlavePHY + MemoryModel backend, then:

```
r = await cocotb.external(prog)() # prog is an UNMODIFIED
pumice_master routine
```

`make_uart_channel(dut, clock, clks_per_bit)` returns a `CocotbUartChannel` that drives the UART pins; the synchronous host program runs in a worker thread under `cocotb.external`, and the channel's read/write are `cocotb.function` wrappers that step the sim. The baud is lowered for sim (`UART_BAUD = 6_250_000` → `CLKS_PER_BIT = 16` vs 868 on board) — bit-timing only, the byte stream is unchanged. `make_sim/sim-smoke/sim-simple/sim-level` run the same programs the board runs.

7.3 Match the board's exact DFI config

`GEAR_RATIO = log2(DFI_RATE)` and **must** match the compile-time `DFI_RATE` or reads corrupt. The test threads `TEST_DFI_RATE` / `TEST_GEAR_RATIO` / `TEST_DRAM_BEAT_BYTES` into both the SV parameters=`{...}` and the host via `drv.set_dfi_phase(...)`. The board is built at `DFI_RATE = 4` (`GEAR = 2`, AXI 64 / DRAM beat 32); a rate-2 / GEAR-1 sim will **pass while the board fails** — it skips the very paths that break. Named variants (`..._smoke_rate4`, `..._smoke_rate4_x16`, `..._pagehit_rate4_x16`, ...) exist to run the board's real geometry.

7.4 What the sim can and cannot reproduce

- **CAN:** everything digital — the controller/engines, the bridge RTL, DFI-level handshakes, register logic. This is where it earns its keep.
- **CANNOT:** analog PHY behavior. Xilinx SERDES/IDELAY in `a7ddrphy` do not simulate, so read-eye / DQS-gate / pin-LOC problems are silicon-only.

8 Troubleshooting

8.1 Current board state

Sim timing is closed; on-board DDR2 runs but is **gated by PHY calibration**. A clean read-leveling eye is found only at `t_phy_wrlat = 0` (bit-slip 0, taps 0–11, centre tap 5). The default of `t_phy_wrlat = 4` used by `--simple` / `--char` breaks the write path (“no passing tap”).

Known-good board config: `t_phy_wrlat = 0`, `t_rddata_en = 6`, `rd_phase = 0`, `rddata_delay = 8`, `cmd_delay = 1`.

After the runtime page-policy fix, OPEN vs CLOSE page policy moved read throughput from ~12.7 MB/s to ~112 MB/s (8.8×) and write to ~44 MB/s (3.5×). For reference, LiteDRAM on the same board reaches ~600 MB/s peak (300 MT/s memtest).

8.2 Sticky errors look like a wedge

`STATUS.any_error` is **sticky** and is cleared by `CTRL.clear_stats`, **not** `soft_reset`. A single failed read latches `rd_error`; every subsequent engine wait then returns `False` and looks like a hardware wedge that isn't one. Clear stats between phases. Note `pumice_master.wait_engine()` waits on **one** engine (write-then-read is phased); a “wait for both” would hang.

8.3 Known bugs / bounds

Known issues and bounds

Symptom	Cause / workaround
Read engine wedges at ~4790 txns	<code>rd_cmd_cam</code> ceiling — keep board runs $\text{txn} \leq 1024$ for clean completion; the <i>perf rate</i> is still valid at scale
<code>row_major</code> wedges immediately	every burst is a row-miss — needs the wedge fix before it can be characterized
Intermittent 1–11 beat mismatches at scale	clean at $\text{txn} \leq 1024$
<code>col_major</code> stride overflow	stride $16 \text{ KB} \times \text{txn} > 8192$ exceeds 128 MB → address alias; host bounds <code>col_major</code> to $\text{txn} \leq 8192$
<code>synth_mask_obs / lookahead_max_obs</code> read 0	init never set <code>CTRLR_CAP</code> — advertise capability first

8.4 Cannot find the board

The USB-UART re-enumerates across reboots/replugs. `--port auto` (default) probes each `/dev/ttyUSB*`, reads `BUILD_ID`, and picks the one answering `0x44445232`. If none respond it raises “no pumice DDR2 char harness responded ... is the board powered and programmed?” — confirm the bitstream is flashed and the FTDI cable is free.

8.5 a7ddrphy is not simulatable

Verilator cannot model `OSERDESE2` / `ISERDESE2` / `IDELAYE2`, so sim connects at the DFI level with `DFISlavePHY` + `MemoryModel`. Read-eye / DQS-gate / pin-LOC problems are silicon-only. Regenerating the real PHY requires **Python 3.10 only** (migen's CSR-name tracer breaks on 3.11/3.12):

```
uv venv --python 3.10 /tmp/litex-venv310 && source /tmp/litex-venv310/bin/activate
uv pip install migen "litex==2024.12" "litedram==2024.12" pyyaml
```

```
cd projects/fpga-systems/NexysA7/pumice/build-perf
python3 bin/elaborate_a7ddrphy.py --out
rtl-vivado/a7ddrphy/a7ddrphy_generated.v
python3 bin/elaborate_a7ddrphy.py --dump-csr-map      # CSR offsets
for firmware
```

8.6 debug_sram trace ring is disabled on this build

DEBUG_SRAM_WORDS was shrunk 32768 → 512 — the full ring needed ~44 K distributed-RAM cells (2.4× over the 100T budget) and blocked place_design. The 256-KB address *window* is unchanged (accesses > 2 KB alias into the ring); the ring is effectively unused here. Raise it on a larger device.

8.7 LiteDRAM baseline flow

flows-litedram-uart is wired and lint-clean but board bring-up is gated. The default ./regen.sh builds an empty BIOS ROM (-no-compile-software) so the core never asserts init_done — use ./regen.sh -bios (needs riscv-gcc). The host variant (harness_csr at base 0, no pumice APB cfg) is still TODO.

9 Measured Results

Every number in this chapter is from the Nexys A7, not simulation: 75 MHz controller clock, DDR2-300, BL4 on the x16 MT47H64M16, open_page policy, the 4+4 generator bitstream. **Peak is 600 MB/s** — 8 bytes per controller cycle. Measured 2026-09-14.

The figures and the raw records are in the repository, so nothing here has to be taken on trust: build-perf/reports/bank_gap_sweep.json (192 records), axlen_sweep.json, outstanding_sweep.json, and the plots under build-perf/reports/plots/.

9.1 8.1 Streaming bandwidth, and why short bursts fall short

Reads and writes both reach about 95% of peak once bursts are long enough. They do not at short burst lengths, and the reason is **latency, not overhead**.

Read bandwidth against AXI burst length, at 8 bursts outstanding

AxLEN	read MB/s	% peak	read latency (cycles)	Little's-law prediction
1	98.7	16.4%	50.6	93.1
2	196.6	32.8%	48.0	192.0
4	367.9	61.3%	50.0	355.5

AxLEN	read MB/s	% peak	read latency (cycles)	Little's-law prediction
8	575.0	95.8%	49.8	570.0
16	575.8	96.0%	96.0	570.0

The model is $\min(\text{outstanding} \times \text{AxLEN} / (\text{latency} + \text{AxLEN}), 0.95) \times \text{peak}$. With 8 bursts in flight against a ~49-cycle read, a one-beat burst simply cannot fill the pipe — the requester runs out of things to have asked for.

That is an inference from one curve, so it was tested directly from the other axis: hold AxLEN and sweep the number of bursts in flight instead.

The short-burst shortfall recovers completely when more reads are in flight

AxLEN	at 8 outstanding	best reached	at
1	98.7 MB/s (16.4%)	390.6 MB/s (65.1%)	32, still climbing
2	196.6 MB/s (32.8%)	573.1 MB/s (95.5%)	24
4	367.9 MB/s (61.3%)	574.6 MB/s (95.8%)	16
8	575.0 MB/s (95.8%)	575.6 MB/s (95.9%)	12

This is the decisive result. AxLEN 2 and 4 reach the *same* ceiling as AxLEN 8 once enough reads are in flight. A per-transaction penalty could not behave that way — it would scale with the number of transactions and never go away. So nothing in the controller's datapath limits short bursts; the read latency and the requester's budget are the whole story. AxLEN 1 needs roughly 49 bursts in flight and the harness ceiling is 32, which is why it alone is still climbing rather than a different mechanism.

The knees confirm the shape too: they scale as $1/\text{AxLEN}$ exactly — none inside 32 at AxLEN 1 (model 49), 32 at AxLEN 2 (model 24.5), 24 at AxLEN 4 (model 12.3), 12 at AxLEN 8 (model 6.2). The *constant* runs 1.3-2x above the model, which is an open question; note the sweep steps 1, 2, 4, 8, 12, 16, 24, 32, so a knee is located only to within one step.

9.2 8.2 How much idle the controller absorbs

The gap sweep runs N writers and N readers **concurrently**, one bank apiece, inserting 0 to 15 idle clocks between bursts, across three address orders and at 4+4 / 3+3 / 2+2 / 1+1 engines. It answers a different question from the bandwidth sweeps: not “how fast can it go” but “how much slack does it have”.

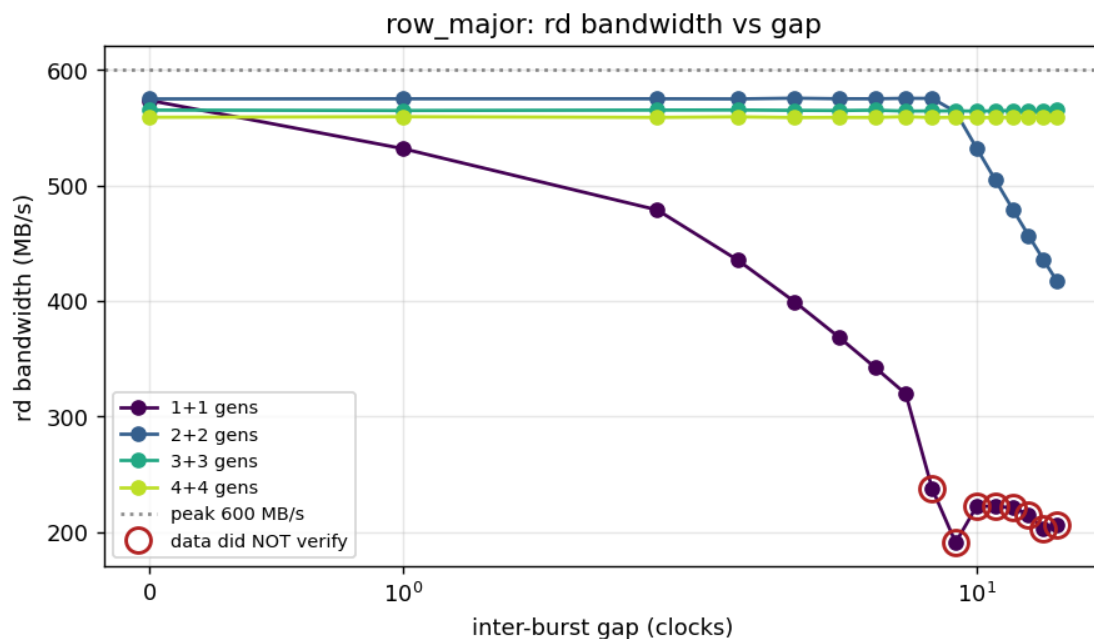
The knee is the largest gap whose read bandwidth is still within 3% of the gap-0 value.

Gap knee by address order and generator count

address order	4+4	3+3	2+2	1+1
cacheline	15	15	9	0
row-major	15	15	9	0
col-major	15	15	15	15

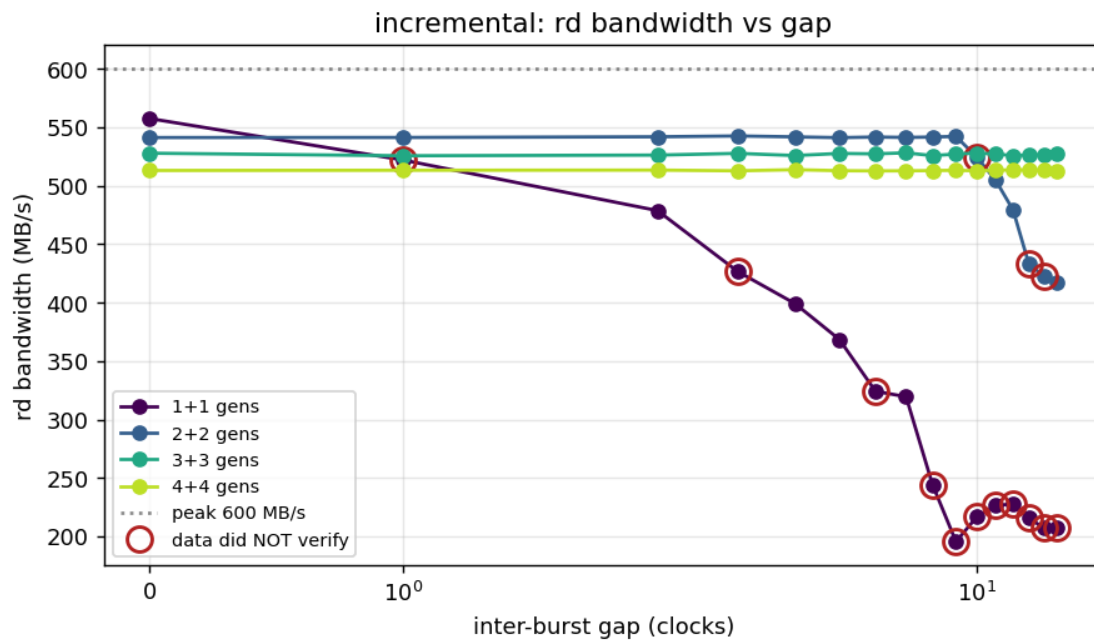
Read bandwidth from gap 0 to gap 15: 1+1 row-major 574 -> 206 MB/s (36% retained), 2+2 575 -> 417 (73%), 3+3 565 -> 565, 4+4 559 -> 559.

9.2.1 Figure 8.1: Read Bandwidth vs Gap, Row-Major



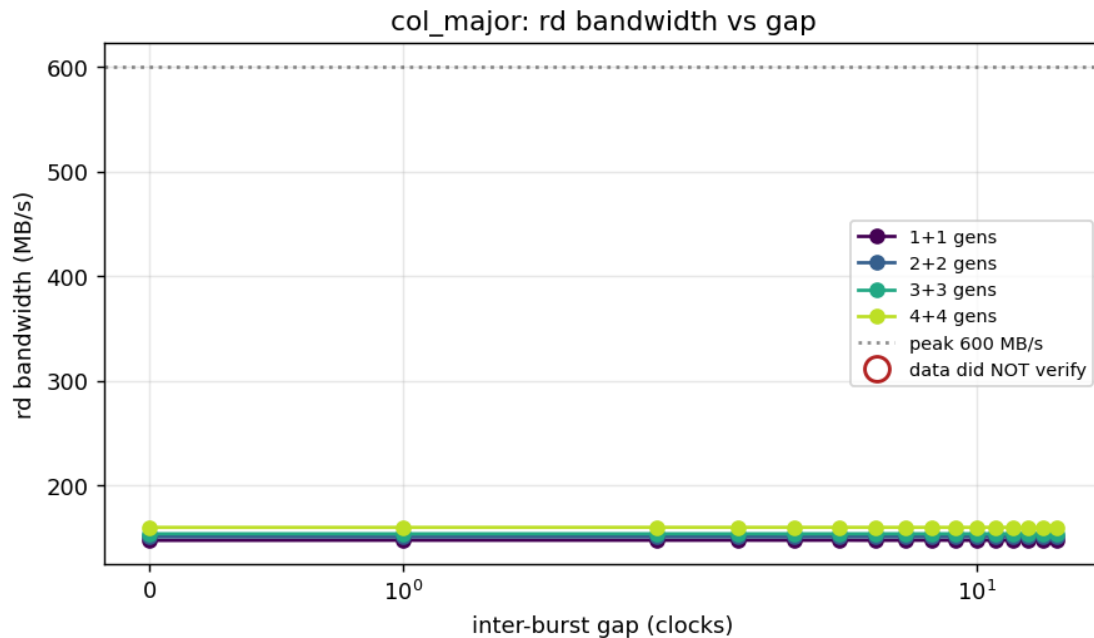
Read bandwidth against inter-burst gap, row-major, one line per generator count

9.2.2 Figure 8.2: Read Bandwidth vs Gap, Cacheline



Read bandwidth against inter-burst gap, cacheline order

9.2.3 Figure 8.3: Read Bandwidth vs Gap, Col-Major



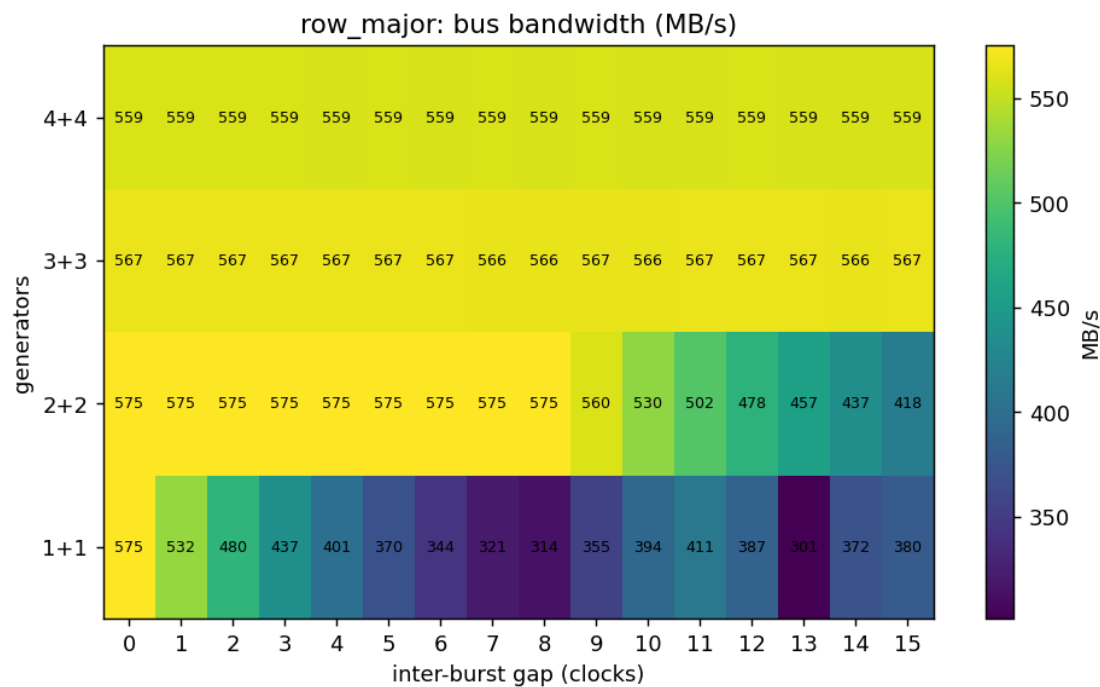
Read bandwidth against inter-burst gap, col-major order

Reading these curves. Two things look wrong at first glance and are not:

The high generator counts are flat. A gap only bends the curve once aggregate demand falls **below** what the controller can deliver. At 4+4, gap 15 still leaves each generator at roughly a 52% duty cycle, and four of those together still out-demand the controller — so nothing moves. A flat curve there is a result, not a broken axis. It also means the 4-bit gap field cannot inject enough idle to starve four generators: at high counts the lever is the outstanding dial, not the gap.

Col-major is flat at every generator count. It is page-miss bound at about 150 MB/s, 25% of peak. The DRAM is the limit, so generator pacing never becomes the binding constraint. Compare it against row-major’s 575 MB/s on the same silicon and the same settings: **a 3.8x spread from the address pattern alone**, which remains the single most useful fact in this guide. Tune the address map before tuning anything else.

9.2.4 Figure 8.4: Bus Bandwidth Across Gap and Generator Count



Heatmap of total bus bandwidth against gap and generator count, row-major

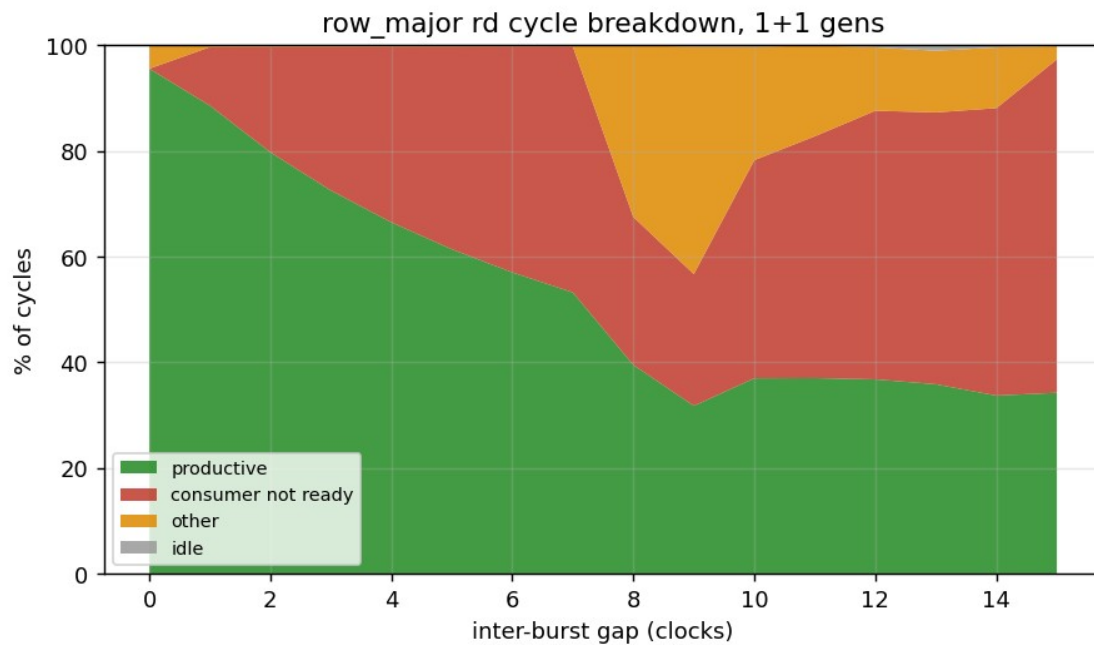
9.3 8.3 Where the cycles went

Bandwidth says a point was slow. The cycle classification says *who stopped*, which is the difference between a finding and a number.

Read cycle breakdown, row-major, 1+1 generators

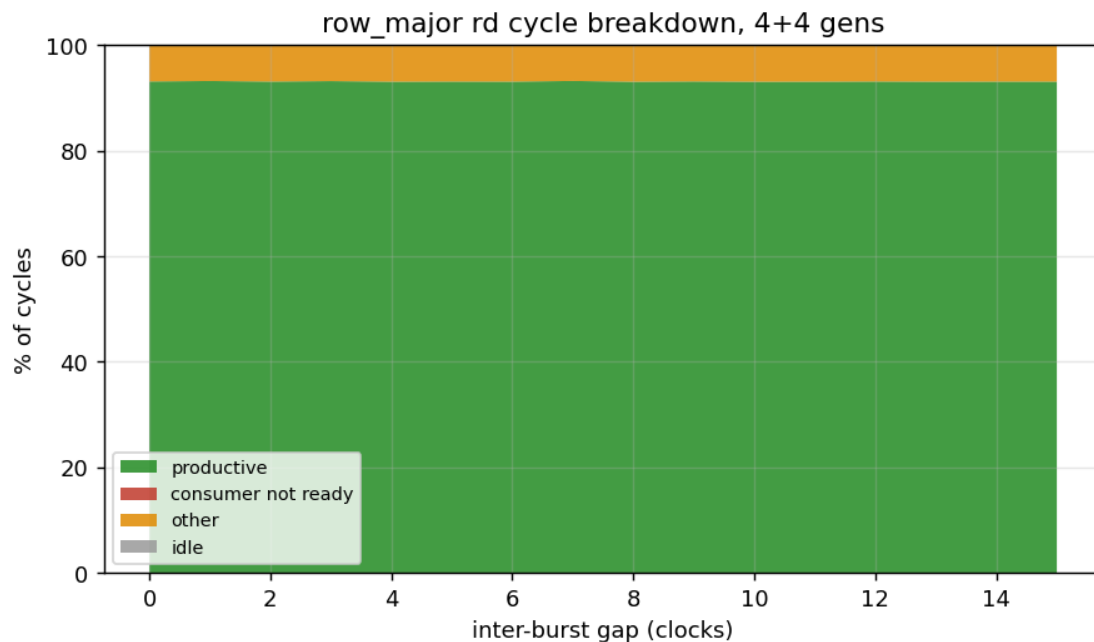
gap	read MB/s	productive	consumer not ready	other
0	574	95.6%	0.0%	4.4%
4	399	66.5%	33.2%	0.2%
8	237	39.5%	28.0%	32.5%
15	206	34.3%	63.1%	2.6%

9.3.1 Figure 8.5: Cycle Breakdown at 1+1 — the Bend and its Cause



Read cycle breakdown against gap at one generator per direction

9.3.2 Figure 8.6: Cycle Breakdown at 4+4 — Saturated at Every Gap



Read cycle breakdown against gap at four generators per direction

At 1+1 the productive fraction falls from 95.6% to 34.3% while “consumer not ready” climbs from nothing to 63.1%. That is the engine’s own inter-burst gap deasserting `rready`, so **the bandwidth went to generator-induced idle, not to anything the controller did**. At 4+4 the productive fraction sits at 93.1% regardless of gap — the controller is saturated everywhere on that axis, which is the same fact Figure 8.1 shows as a flat line, seen from the cause side.

Two traps in this instrument, both of which produced published nonsense before they were caught.

The bus meter **free-runs** from `clear_stats()` until it is read, so its own total spans the host’s UART chatter as well as the transfer — 7.2M cycles against a 16.7k-cycle window on a 1+1 point, a factor of 430. Fractions taken against that total made a point moving 95% of peak report “0.2% productive, 99.8% starvation”. The *counts* are sound — productive lands on exactly the beats moved — so the fix is the denominator: normalise against the timer’s first-to-last stamp for that direction. The meter’s own starvation bucket is where the host idle lands and is not reported as a fraction at all.

On a **read**, the engine is the *consumer*, so its own pacing gap shows up as `rvalid && !rready` — the bucket a write would call backpressure. It is generator-induced idle, the opposite of the controller refusing. The label

in Figure 8.5 says “consumer not ready” for exactly this reason; do not read the read and write breakdowns with the same vocabulary.

9.4 8.4 Integrity, and one configuration that must not be quoted

All 192 gap-sweep points completed. **22 of them returned mismatching data**, and they are not scattered: they are a specific configuration, tracked as PUMICE-037.

Concurrent read+write with the **reader’s** gap at 8 or above returns wrong data, and when the two address ranges overlap it leaves genuinely corrupted cells behind. The boundary was narrowed by elimination on the board, each line its own run from a verified-clean 128 MiB prefill:

PUMICE-037 — narrowing the failing configuration

configuration	result
prefill, then read every bank read-only, all three orders	clean
reader alone, gap 0-15	clean at every gap
writer alone, gap 0-15	cells clean
writer gap 0-15, reader gap 0	clean at every writer gap
reader gap 0-7 with a concurrent writer	clean
reader gap >= 8 with a concurrent writer	1.5k-7k of 32000 beats wrong
4+4 concurrent at gap 0 and gap 4	clean, cells clean

Neither engine alone does it at any gap, and the writer’s own gap never does it. **Everything in this chapter is at gap 0-7 and is unaffected**, but no gap ≥ 8 concurrent point may be quoted as a clean operating point. The plots ring those points, and `plot_bank_gap.py` refuses any record file whose bandwidth exceeds its own stored ceiling.

Whether the defect is in the controller or in the harness read engine is not yet settled. The stray-beat counter is zero on all 22 failing points, which rules out the read engine’s stray-drain path and points at the data itself; against that, a read engine cannot corrupt cells, and cells are corrupted. The next step is a simulation reproduction with waves — and the reason this reached the board at all is that both gap-bearing simulation suites drain the writer before starting the reader, so **concurrency with a gap has never been simulated**.

9.5 8.5 Reproducing these numbers

```
cd build-perf
make bitstream && make program
python3 bin/axlen_sweep.py           # 8.1, first table
python3 bin/outstanding_sweep.py     # 8.1, second table
python3 bin/bank_gap_sweep.py        # 8.2-8.4, ~12 minutes
python3 bin/plot_bank_gap.py reports/bank_gap_sweep.json
```

Leave PREFILL at its default (point). It re-fills the whole device before every measurement so that points are independent; without it, a point that corrupts cells is inherited by every later point that reads the region and reported as *that* point's result. Chapter 3 lists the rest of the pitfalls, and docs/DDR2_BANDWIDTH_MEASUREMENT.md carries the full treatment.